Projekt 3 – sieci bayesowskie z niepełnych danych

Termin oddania: 17/18 listopada 2026.

Opis problemu

Uczelnia zebrała ankiety o studentach: trudność kursu, zdolności studenta, ocenę, wynik testu SAT, list polecający oraz, zależnie od zestawu, m.in. motywację, frekwencję, czas nauki, stypendium, ofertę pracy. Dane są **niepełne**: w wielu wierszach brakuje niektórych wartości, np. zdolności zmierzono tylko części studentów. Wiemy też, jak zwykle wyglądają takie zależności (sieć „student" znana z zajęć: $D \to G \leftarrow I$, $G \to L$, $I \to S$), ale opinia eksperta o pozostałych zależnościach może być niedokładna.

Celem projektu jest zbudowanie z tych danych **sieci bayesowskiej** w bibliotece pgmpy i odpowiadanie na zapytania do modelu: o rozkłady warunkowe, o najbardziej prawdopodobne konfiguracje (MAP), a na najwyższy próg także o warunki, których biblioteka nie obsługuje (np. „ocena A lub B, i dobry SAT lub dobry list"). Zalecany jest Python i pgmpy (`pip install pgmpy`); rozwiązanie wzorcowe powstało na wersji 1.1.2. Część zadań wymaga napisania kodu samodzielnie, bo gotowe funkcje biblioteki mają ograniczenia, które trzeba poznać (patrz „Wskazówki").

Zmienne są dyskretne, a tablica rozkładu łącznego ma co najwyżej kilka tysięcy komórek. Dlatego wynik próbkowania zawsze da się porównać z wynikiem dokładnym.

Pojęcia

Model i zapytania:

- **CPD** (*conditional probability distribution*, rozkład prawdopodobieństwa warunkowego) to tablica $P(X \mid \mathrm{Pa}(X))$ przypisana do zmiennej X o rodzicach $\mathrm{Pa}(X)$. Dla każdej konfiguracji rodziców zawiera rozkład zmiennej X, sumujący się do 1. Zwykle jednej konfiguracji rodziców odpowiada wiersz, a jednej wartości X kolumna; uwaga: `TabularCPD` w pgmpy używa układu odwrotnego (wiersze to wartości X, kolumny to konfiguracje rodziców). Rozkład łączny sieci jest iloczynem wszystkich CPD. Zmienną razem z jej rodzicami nazywamy **rodziną**.
- **MAP** (*maximum a posteriori*) to w tym projekcie rodzaj zapytania: najbardziej prawdopodobna łączna konfiguracja zmiennych Q przy świadectwie $E = e$, czyli $\arg\max_q P(Q = q \mid E = e)$. Nie musi być równa zestawieniu najbardziej prawdopodobnych wartości każdej zmiennej osobno.

Uczenie parametrów:

- **MLE** (*maximum likelihood estimation*, estymacja największej wiarygodności) wybiera parametry, przy których dane są najbardziej prawdopodobne. Dla pełnych danych są to względne częstości: $\hat{P}(X = x \mid \mathrm{Pa}(X) = u) = N(x, u)/N(u)$, gdzie N oznacza liczbę wierszy o podanych wartościach.
- **EM** (*expectation-maximization*) to iteracyjna metoda szukania parametrów o dużej wiarygodności, gdy część wartości jest nieznana. **E-krok**: przy bieżących parametrach oblicz rozkład a posteriori brakujących wartości w każdym wierszu i na jego podstawie oczekiwane zliczenia. **M-krok**: oszacuj parametry jak w MLE, ale z oczekiwanych zliczeń. Log-wiarygodność danych obserwowanych nie maleje między iteracjami, ale metoda może się zatrzymać w optimum lokalnym.

Mechanizmy powstawania braków (pojęcia potrzebne w zadaniu dla chętnych; przykłady dotyczą ankiety o dochodach):

- **MCAR** (*missing completely at random*): to, czy wartość jest brakująca, nie zależy od żadnych danych. Przykład: część ankiet zgubiono przypadkowo.
- **MAR** (*missing at random*): brak może zależeć od wartości obserwowanych, ale przy ich znajomości nie zależy już od samej ukrytej wartości. Przykład: młodsze osoby rzadziej podają dochód, a wiek jest znany zawsze.
- **MNAR** (*missing not at random*): brak zależy od samej ukrytej wartości także po uwzględnieniu wartości obserwowanych. Przykład: osoby o wysokich dochodach częściej ich nie podają.

Uczenie struktury:

- **BIC** (*Bayesian information criterion*) to ocena grafu G, która nagradza dopasowanie do danych i karze za liczbę parametrów: $\mathrm{BIC}(G) = \log P(\text{dane} \mid G, \hat{\theta}) - \frac{\log N}{2} k$, gdzie $\hat{\theta}$ to parametry z MLE, N to liczba wierszy, a k to liczba swobodnych parametrów. W pgmpy wyższa wartość oznacza lepszy graf (w części literatury stosuje się inny znak lub skalę). Wynik jest sumą ocen rodzin, więc po zmianie jednej krawędzi wystarczy przeliczyć tylko zmienione rodziny.
- **PDAG** (*partially directed acyclic graph*) to graf częściowo skierowany: część krawędzi ma kierunek, a część nie. Służy do zapisu **klasy równoważności Markowa**, czyli grafów skierowanych o tym samym szkielecie i tych samych v-strukturach, które kodują te same niezależności. Krawędź nieskierowana oznacza, że jej kierunku nie da się ustalić z samych danych. Aby oszacować CPD, trzeba najpierw wybrać graf skierowany (DAG) z tej klasy.

Porównywanie rozkładów:

- **TV** (*total variation distance*, odległość całkowitej zmienności) mierzy, jak bardzo różnią się dwa rozkłady p i q na tym samym skończonym zbiorze wartości: $\mathrm{TV}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|$. Przyjmuje wartości od 0 (rozkłady identyczne) do 1 (rozkłady o rozłącznych nośnikach). Jest to też największa możliwa różnica prawdopodobieństwa tego samego zdarzenia w obu rozkładach. Przykład: dla $p = (0{,}5;\ 0{,}3;\ 0{,}2)$ i $q = (0{,}4;\ 0{,}4;\ 0{,}2)$ otrzymujemy $\mathrm{TV} = \frac{1}{2}(0{,}1 + 0{,}1 + 0) = 0{,}1$. Dla zapytania o kilka zmiennych suma obejmuje wszystkie ich łączne konfiguracje.

Formaty danych

Zestaw danych to katalog z pięcioma plikami. Do projektu dołączono katalog `przyklad/` (model z tutoriala pgmpy) do sprawdzenia implementacji.

Plik `opis.json` opisuje zmienne i wiedzę dziedzinową. Pole `meta` jest ignorowane, a inne pola niż podane poniżej są niedozwolone:

```
{
  "meta": {"description": "Przykład"},
  "variables": {
    "D": {"description": "trudność kursu", "states": ["Easy", "Hard"]},
    "I": {"description": "zdolności studenta", "states": ["Dumb", "Intelligent"]},
    "G": {"description": "ocena z kursu", "states": ["A", "B", "C"]},
    "S": {"description": "wynik testu SAT", "states": ["Bad", "Good"]},
    "L": {"description": "list polecający", "states": ["Bad", "Good"]}
  },
  "expert_edges": [["D", "G"], ["I", "G"], ["G", "L"], ["I", "S"]],
  "required_edges": [["D", "G"], ["I", "G"], ["G", "L"], ["I", "S"]],
  "forbidden_edges": [["D", "S"], ["G", "S"]],
  "tiers": [["D", "I"], ["G", "S"], ["L"]]
}
```

- `expert_edges` to **graf ekspercki**. Może nie zawierać niektórych prawdziwych zależności i może zawierać zależności, których w rzeczywistości nie ma.
- `required_edges` i `forbidden_edges` to krawędzie na pewno obecne i na pewno nieobecne.
- `tiers` to warstwy czasowe: zmienna z późniejszej warstwy nie może być przyczyną zmiennej z wcześniejszej. Warstwy nie ustalają kierunków krawędzi w obrębie jednej warstwy.
- Nazwy zmiennych i stanów składają się z liter, cyfr i znaku `_`, zaczynają się literą i nie mogą być słowami kluczowymi Pythona ani słowami `P`, `MAP`, `atleast`, `atmost`. Stany zmiennej mają ustaloną kolejność.

Plik `dane.csv` zawiera dane uczące: wiersz nagłówka z nazwami zmiennych (w dowolnej kolejności) i po jednym studencie w wierszu. Puste pole oznacza brak wartości:

```
D,I,G,S,L
Easy,Dumb,C,Bad,Bad
Hard,,C,Bad,
```

Plik `test.csv` ma ten sam format, ale nie ma w nim braków. Służy wyłącznie do oceny modeli. Nie wolno go używać do uczenia.

Plik `zapytania.txt` zawiera po jednym zapytaniu w linii. Linie puste są pomijane, a znak `#` rozpoczyna komentarz do końca linii. Zapytanie `P(...)` pyta o rozkład łączny podanych zmiennych, a `MAP(...)` o ich najbardziej prawdopodobną łączną konfigurację. Po znaku `|` może wystąpić świadectwo, czyli równości oddzielone przecinkami:

```
P(G)
P(G, L | S==Good)
MAP(I, D | G==A, S==Good)
```

Plik `ograniczenia.txt` (zadania na 8 pkt) ma ten sam format, ale po znaku `|` może wystąpić dowolna **formuła**. Formuła jest poprawnym wyrażeniem Pythona (nazwy stanów pisze się bez cudzysłowów), więc nie trzeba pisać parsera: analizę składni wykonuje moduł `ast` z biblioteki standardowej (patrz „Wskazówki").

zapis	znaczenie
`X==s`, `X!=s`	zmienna ma stan s; ma stan inny niż s
`X in {s1, s2}`	zmienna ma jeden z podanych stanów
`not f`, `f and g`, `f or g`	negacja, koniunkcja, alternatywa
`atleast(k, f1, ..., fn)`	co najmniej k z podanych formuł jest prawdziwych
`atmost(k, f1, ..., fn)`	co najwyżej k z podanych formuł jest prawdziwych
`f, g`	przecinek na najwyższym poziomie oznacza koniunkcję

Priorytety są takie jak w Pythonie: `not`, potem `and`, potem `or`. Można używać nawiasów. Implikację „jeśli f, to g" zapisuje się jako `not f or g`. Zmienna zapytania nie może występować w warunku. Przykłady:

```
P(I | G in {A, B} and (S==Good or L==Good))
P(D | not S==Good or G==A)
P(I | atleast(2, G==A, L==Good, S==Good))
```

Zestawy do eksperymentów. Każda grupa wybiera numer z pliku „numery grup.xlsx" odpowiadający numerowi jednego z członków grupy i pracuje **wyłącznie** ze swoim katalogiem `zestawy/kXX/` z archiwum `zestawy.zip` (np. `zestawy/k07/` dla numeru 7). Każdy z 40 zestawów pochodzi z innego wariantu sieci „student": ma dodatkowe zmienne, dodatkowe krawędzie i inne parametry, a także inną liczbę wierszy i inny **mechanizm powstawania braków**. Prawdziwego modelu nie podajemy. W raporcie podaj swój numer.

Wyniki kontrolne dla katalogu `przyklad/`. Wyniki MLE powinny się zgadzać co do podanych cyfr. Wyniki EM mogą się różnić o około 0,002, zależnie od kryterium stopu.

```
P(G)                             MLE: A 0.349, B 0.317, C 0.334     EM: A 0.352, B 0.299, C 0.348
P(G | D==Easy, I==Intelligent)   MLE: A 0.886, B 0.079, C 0.035     EM: A 0.891, B 0.072, C 0.037
P(I | G==A, S==Good)             MLE: Dumb 0.023, Intelligent 0.977 EM: Dumb 0.021, Intelligent 0.979
P(I | G==C)                      MLE: Dumb 0.924, Intelligent 0.076 EM: Dumb 0.916, Intelligent 0.084
P(I | G==C, D==Hard)             MLE: Dumb 0.910, Intelligent 0.090 EM: Dumb 0.894, Intelligent 0.106
MAP(G | D==Easy, I==Intelligent) MLE: G=A (P = 0.886)               EM: G=A (P = 0.891)
MAP(D, I | G==C, L==Bad)         MLE: D=Hard, I=Dumb (P = 0.571)    EM: D=Hard, I=Dumb (P = 0.549)

Warunki w modelu MLE, P(warunek) i rozkład zmiennej zapytania:
P(I | G in {A, B} and (S==Good or L==Good))    0.54207, (0.495, 0.505)
P(D | not S==Good or G==A)                     0.91332, (0.620, 0.380)
P(I | atleast(2, G==A, L==Good, S==Good))      0.36560, (0.325, 0.675)
P(D | G==C and L==Good and S==Good)            0.00032, (0.296, 0.704)

Ocena na test.csv: średni log P(wiersz), trafność przewidywania G z pozostałych zmiennych:
```

```
MLE             -2.7838, 0.704
EM              -2.7843, 0.702
```

`HillClimbSearch` z oceną BIC na wierszach pełnych z wiedzą ekspercką daje tu graf ekspercki.

Zadania

Progi punktowe są kumulatywne: na 6 pkt trzeba wykonać także zadania na 4 pkt, a na 8 pkt – zadania na 4 i 6 pkt.

Zadania na 4 punkty (model z grafu eksperckiego i zapytania)

1. Napisz program wczytujący zestaw danych. Opis zestawu i zapytania reprezentuj za pomocą wyodrębnionych klas. Program ma odrzucać niepoprawne pliki z czytelnym komunikatem wskazującym miejsce błędu (numer linii albo ścieżkę w dokumencie JSON), np.: niepoprawny JSON, nieznane pole, zmienna z jednym stanem, powtórzony stan, cykl w `expert_edges`, nieznana kolumna lub brakująca kolumna w CSV, zła liczba pól w wierszu, nieznany stan, brak wartości w `test.csv`, zapytanie o nieznaną zmienną, zapytanie, w którym ta sama zmienna jest celem i warunkiem. W raporcie podaj liczbę wierszy, liczbę wierszy w pełni obserwowanych i odsetek braków dla każdej zmiennej.
2. Zbuduj sieć o strukturze grafu eksperckiego i oszacuj parametry estymatorem największej wiarygodności (`DiscreteMLE`). Każda tablica CPD jest liczona z wierszy, w których cała rodzina (zmienna i jej rodzice) jest obserwowana. Podaj największą tablicę CPD. Policz, ile konfiguracji jej rodziców nie ma ani jednego wiersza, i sprawdź, co biblioteka wpisuje dla takich konfiguracji.
3. Zaimplementuj odpowiadanie na zapytania z pliku `zapytania.txt` za pomocą `VariableElimination` (P i MAP). W raporcie podaj odpowiedzi.
4. Oceń model na `test.csv`: średni log P(wiersz), liczba wierszy, którym model przypisuje $P = 0$, oraz trafność przewidywania `G` z pozostałych zmiennych. W logarytmie ogranicz P z dołu przez 10^{-10}. Porównaj z modelem bez krawędzi i z klasyfikatorem zawsze wskazującym najczęstszą ocenę. Skomentuj wyniki oraz na przykładach ze swojego zestawu pokaż: rozumowanie przyczynowe, diagnostyczne i *explaining away*. Dla kilku trójek zmiennych porównaj d-separację (`is_dconnected`) z liczbami: gdy zmienne są d-oddzielone, obserwacja jednej nie może zmieniać rozkładu drugiej.

Zadania na 6 punktów (EM, uczenie struktury)

1. Zaimplementuj **algorytm EM** dla sieci o danej strukturze i danych z częściowymi brakami. Implementacja ma działać tylko dla **danych dyskretnych**, czyli zmiennych o skończonej liczbie stanów podanych w `opis.json`; zmiennych ciągłych nie trzeba obsługiwać. Nie można tu użyć `DiscreteEM` (patrz „Wskazówki"), co pokaż w raporcie. Pseudokod:

```
wejście: rodzice Pa(X) każdej zmiennej X, wiersze danych z brakami
wyjście: tablice CPD

start: dla każdej zmiennej X i konfiguracji rodziców u
    P(X = x | Pa(X) = u) := losowy rozkład
    (albo znormalizowane zliczenia z wierszy z pełną rodziną, powiększone o 1)
powtarzaj:
    # E-krok: oczekiwane zliczenia przy bieżących CPD
    N[X][x, u] := 0 dla wszystkich X, x, u
    L := 0
    dla każdego wiersza r:
        H := zmienne brakujące w r
        dla każdej konfiguracji h zmiennych H:
            p[h] := iloczyn wszystkich CPD dla wiersza r uzupełnionego przez h
        p_r := suma p[h] po wszystkich h          # P(obserwacje z wiersza r)
        L := L + log(p_r)
        dla każdej konfiguracji h zmiennych H:
```

```
            dla każdej zmiennej X:
                (x, u) := wartość X i wartości Pa(X) w wierszu r uzupełnionym przez h
                N[X][x, u] := N[X][x, u] + p[h] / p_r
    # M-krok: częstości z oczekiwanych zliczeń
    dla każdej zmiennej X i konfiguracji rodziców u:
        S := suma N[X][x, u] po wszystkich x
        jeśli S > 0: P(X = x | Pa(X) = u) := N[X][x, u] / S dla każdego x
        wpp.:        P(X = x | Pa(X) = u) := 1 / (liczba stanów X) dla każdego x
aż |L - poprzednie L| < 1e-6 * (liczba wierszy) albo do limitu iteracji
```

Sprawdź, że log-wiarygodność danych obserwowanych L nie maleje między iteracjami, i uruchom EM z co najmniej 5 losowych punktów startowych. Porównaj: MLE tylko na pełnych wierszach, MLE po rodzinach i EM (log-wiarygodność testowa, rozkłady brzegowe a częstości w `test.csv`, odpowiedzi na zapytania). Wyjaśnij, skąd biorą się różnice.

2. Naucz strukturę sieci za pomocą `HillClimbSearch` z oceną BIC, na co najmniej trzy sposoby: na pełnych wierszach bez wiedzy eksperckiej, na pełnych wierszach z wiedzą (`ExpertKnowledge`: warstwy, krawędzie wymagane i zabronione) oraz metodą wykorzystującą wszystkie wiersze, np. pętlą: EM → uzupełnienie każdego braku próbką z rozkładu a posteriori → hill climbing. Dla każdego grafu oszacuj parametry w ten sam sposób i podaj: krawędzie, różnice względem grafu eksperckiego, liczbę parametrów, log-wiarygodność danych uczących i testowych, trafność przewidywania `G`. Skomentuj, które różnice mogą wynikać z równoważności Markowa. Wybierz model końcowy i uzasadnij wybór. Pokaż, które odpowiedzi na `zapytania.txt` zmieniają się najbardziej względem modelu z grafem eksperckim, i wyjaśnij dlaczego.

Zadania na 8 punktów (próbkowanie z ograniczeniami)

Funkcja `BayesianModelSampling.rejection_sample` przyjmuje tylko koniunkcję równości, więc warunków z pliku `ograniczenia.txt` nie da się jej przekazać. W tych zadaniach próbkowanie ma być Twoją implementacją; biblioteka służy jedynie do porównania.

1. Zaimplementuj wartościowanie formuł na paczkach próbek (składnię analizuje `ast.parse`; gotowy szkielet jest we „Wskazówkach") i **próbkowanie z odrzucaniem dla dowolnej formuły**. Niepoprawną formułę należy odrzucić z czytelnym komunikatem. Próbkowanie w przód ma być własne (zmienne w porządku topologicznym, losowanie z CPD) i wektorowe, tzn. całe paczki próbek naraz, a nie pętla po próbkach w Pythonie. Program ma liczyć wszystkie zużyte próbki i zgłaszać przekroczenie limitu ich liczby, zwracając to, co udało się zebrać. Dla każdego zapytania z `ograniczenia.txt` w modelu końcowym podaj: wynik, odsetek akceptacji, liczbę próbek i czas.
2. Sprawdź poprawność. Policz wynik dokładny i P(warunek) z rozkładu łącznego zwróconego przez `VariableElimination` (patrz „Wskazówki") i podaj odległość całkowitej zmienności (TV, patrz „Pojęcia") między oszacowaniem a wynikiem dokładnym. Warunek może mieć w Twoim modelu prawdopodobieństwo 0; program ma to wtedy zgłosić. Na zwykłym świadectwie porównaj swój sampler z `rejection_sample` z pgmpy. Dla jednego zapytania pokaż, jak błąd zależy od liczby akceptacji n ($n \in \{10^2, 10^3, 10^4, 10^5\}$, co najmniej 20 powtórzeń), i jak ma się to do $1/\sqrt{n}$. Jak liczba potrzebnych próbek zależy od P(warunek)?

Zadania dla chętnych (nieobowiązkowe)

- Zbadaj mechanizm braków. Testem chi-kwadrat sprawdź, czy brak zmiennej X zależy od wartości zmiennej Y, dla wszystkich par (uwzględnij poprawkę na liczbę testów). Porównaj rozkłady brzegowe z obserwowanych wartości w `dane.csv` z rozkładami w `test.csv`. Oceń, czy braki mogą być MCAR, MAR czy MNAR. Wyjaśnij, czego nie da się rozstrzygnąć na podstawie samego pliku `dane.csv`.
- Zbadaj stabilność uczenia struktury: naucz strukturę na co najmniej 20 próbach bootstrapowych i podaj częstość każdej krawędzi. Które krawędzie modelu końcowego są słabo potwierdzone? Które pary zmiennych są stale połączone, a niestabilny jest tylko kierunek krawędzi?

- Wybierz zmienną X ze swojego zestawu, która ma z `G` wspólną przyczynę, i porównaj $P(G \mid X = x)$ z $P(G \mid do(X = x))$ (`CausalInference`). Kiedy te rozkłady są równe? Co to oznacza, skoro uczenie struktury nie rozstrzyga kierunków niektórych krawędzi?
- Czy EM może usunąć obciążenie przy MNAR? Pokaż na małej symulacji, co się dzieje, gdy słabe oceny są częściej ukrywane.

Wskazówki

Podstawowe elementy pgmpy:

```
from pgmpy.causal_discovery import ExpertKnowledge, HillClimbSearch
from pgmpy.inference import VariableElimination
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.parameter_estimator import DiscreteMLE

model = DiscreteBayesianNetwork()
model.add_nodes_from(states)                  # także zmienne bez krawędzi
model.add_edges_from(edges)
model.fit(data, estimator=DiscreteMLE(state_names=states))   # states: {zmienna: lista stanów}

infer = VariableElimination(model)
factor = infer.query(["G", "L"], evidence={"S": "Good"}, show_progress=False)
infer.map_query(["I", "D"], evidence={"G": "A"}, show_progress=False)

knowledge = ExpertKnowledge(required_edges=required, forbidden_edges=forbidden,
                            temporal_order=tiers)
hc = HillClimbSearch(scoring_method="bic-d", expert_knowledge=knowledge,
                     return_type="dag", show_progress=False)
hc.fit(data.dropna())
edges = list(hc.causal_graph_.edges())
```

Na co uważać:

- `DiscreteEM` **nie** obsługuje częściowych braków: usuwa wiersze z brakami i traktuje jako ukryte tylko całkowicie puste kolumny. Bez takich kolumn wynik to zwykłe MLE na pełnych wierszach, dlatego w zadaniu 6.1 EM trzeba napisać samodzielnie.
- `HillClimbSearch` nie przyjmuje danych z NaN. Domyślnie zwraca PDAG, a do `fit()` potrzebny jest DAG (`return_type="dag"`).
- Wyniki `HillClimbSearch` mogą się różnić między uruchomieniami programu, bo remisy są rozstrzygane w kolejności zależnej od haszowania napisów. Aby wyniki dało się odtworzyć, ustaw zmienną środowiskową `PYTHONHASHSEED`, np. `PYTHONHASHSEED=0 python projekt.py`.
- Kolejność rodziców w CPD (`cpd.variables`) i kolejność stanów (`cpd.state_names`) nie muszą być takie, jak się spodziewasz. Zawsze wyrównuj osie tablic po nazwach zmiennych.
- Nie powielaj wierszy przy uczeniu struktury (np. kilka imputacji połączonych w jeden plik): osłabia to karę BIC względem log-wiarygodności i prowadzi do fałszywych krawędzi.

Zapytania i formuły. W pliku `zapytania.txt` warunek to tylko równości, więc wystarczy podział napisu po `,` i `==`. W zadaniach na 8 pkt formułę analizuje moduł `ast`: `ast.parse` buduje drzewo (priorytety i nawiasy są już uwzględnione), a wystarczy przejść po nim rekurencyjnie. Nie używaj `eval`, bo nie działa z tablicami (`and` na tablicach numpy zgłasza błąd) i wykonałby dowolny kod z pliku. Ten szkielet można wykorzystać wprost; trzeba do niego dodać obsługę błędów:

```
import ast
import re

import numpy as np
```

```
line = "P(I, D | G in {A, B} and (S==Good or L==Good))"
kind, targets, condition = re.fullmatch(r"(P|MAP)\((.*?)(?:\|(.*))?\)", line).groups()
targets = [t.strip() for t in targets.split(",")]
tree = ast.parse(condition.strip(), mode="eval").body    # condition is None: brak warunku


def holds(node, samples, states):
    """Wartość formuły dla paczki próbek: samples[X] - tablica numerów stanów X,
    states[X] - lista nazw stanów X. Wynik to tablica wartości logicznych."""
    if isinstance(node, ast.BoolOp):                     # f and g and ..., f or g or ...
        parts = [holds(v, samples, states) for v in node.values]
        join = np.logical_and if isinstance(node.op, ast.And) else np.logical_or
        return join.reduce(parts)
    if isinstance(node, ast.UnaryOp) and isinstance(node.op, ast.Not):
        return ~holds(node.operand, samples, states)
    if isinstance(node, ast.Tuple):                      # f, g (przecinek)
        return np.logical_and.reduce([holds(v, samples, states) for v in node.elts])
    if isinstance(node, ast.Compare):                    # X==s, X!=s, X in {s1, s2}
        var, op, right = node.left.id, node.ops[0], node.comparators[0]
        names = [e.id for e in right.elts] if isinstance(right, ast.Set) else [right.id]
        mask = np.isin(samples[var], [states[var].index(n) for n in names])
        return ~mask if isinstance(op, ast.NotEq) else mask
    if isinstance(node, ast.Call):                       # atleast(k, ...), atmost(k, ...)
        k = node.args[0].value
        count = np.sum([holds(a, samples, states) for a in node.args[1:]], axis=0)
        return count >= k if node.func.id == "atleast" else count <= k
    raise ValueError(f"niedozwolone wyrażenie: {ast.dump(node)}")
```

Do wyniku dokładnego ta sama funkcja działa na tablicach numerów stanów dla wszystkich komórek rozkładu łącznego (`np.indices`). Budowę drzewa można podejrzeć przez `print(ast.dump(tree, indent=2))`.

Przydatne przy implementacji:

- Wynik dokładny w zadaniu 8.2: `joint = infer.query(list(states), show_progress=False)` zwraca rozkład łączny wszystkich zmiennych. Kolejność osi tablicy `joint.values` podaje `joint.variables`, a kolejność stanów `joint.state_names`. Wyzeruj komórki, w których formuła jest fałszywa; suma pozostałych to P(warunek). Wysumuj pozostałe zmienne i znormalizuj.
- W EM tablicę rozkładu łącznego przy bieżących CPD najprościej zbudować w numpy: iloczyn wszystkich CPD, każda z osiami ustawionymi przez `transpose` i `reshape` tak, aby działał *broadcasting*. Jej wycinek przy wartościach obserwowanych z wiersza to, po normalizacji, rozkład a posteriori brakujących zmiennych. Wiele wierszy się powtarza, więc warto liczyć po unikalnych wzorcach (`np.unique(..., axis=0, return_counts=True)`).
- Do wektorowego losowania z CPD wystarczy dystrybuanta wzdłuż osi stanów: `(u[None, :] >= cdf).sum(axis=0)`, gdzie `cdf[j, i]` to dystrybuanta stanu `j` przy stanach rodziców w próbce `i`, a `u` to liczby losowe z przedziału $[0, 1)$, po jednej na próbkę.

Co oddać

- kod źródłowy z instrukcją uruchomienia;
- raport z wynikami i komentarzem do zadań z wybranego progu (z numerem zestawu);
- projekt należy zaprezentować prowadzącemu w trakcie zajęć i odpowiedzieć na ewentualne pytania;
- *(zalecane)* testy automatyczne, np. sprawdzające, że log-wiarygodność w EM nie maleje oraz że sampler zbiega do wyniku dokładnego.